Transients Without Tears: Automatic In-Place Conversion of Persistent Data Structure Updates in a Dynamic Language

Anonymous Author(s)

Abstract

Persistent-first languages pay a large allocation tax on accumulation loops. Languages like Koka eliminate it with ownership types (Perceus/FBIP); Clojure asks the programmer to opt in manually with transients. We show the optimization can be made *automatic and sound* in a latently-typed language with open-world CLOS dispatch and live redefinition—with no type system—via (1) a tail-position-sensitive *usage classification* that establishes uniqueness-at-death of loop and fold accumulators, (2) a rewriter aligned with the classifier *by construction*, targeting edit-tagged in-place transients, and (3) *world-guarded regions* that keep the optimization sound when definitions change at runtime. The result: 4.1–7.6× speedups on accumulation-bound code, byte-identical outputs on a production-scale event simulator, zero measured cost when disabled or inapplicable, and an honest cost model of when transient conversion pays.

CCS Concepts: • **Software and its engineering → Compilers**; **Dynamic compilers**; • **Theory of computation → Program analysis**; • **General and reference → Performance**.

Keywords: persistent data structures, compilers, optimization, escape analysis, dynamic languages, lisp

ACM Reference Format:
Anonymous Author(s). 2027. Transients Without Tears: Automatic In-Place Conversion of Persistent Data Structure Updates in a Dynamic Language. In *Proceedings of the 48th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '27), June 2027, Somewhere, USA.* ACM, New York, NY, USA, 8 pages. https://doi.org/XXXXXXX.XXXXXXX

PLDI '27, Somewhere, USA

ACM ISBN 978-1-4503-XXXX-X/27/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Languages built on persistent data structures offer powerful guarantees of immutability and thread safety, but they often pay a significant performance penalty known as the "persistence tax." Every update creates a new version of the data structure, allocating new path nodes from the root. In tight accumulation loops, this leads to excessive allocation and garbage collection pressure.

In our language, FOL (a Lisp dialect with persistent data structures), the cost is stark. Compared to handwritten, mutable Common Lisp, our unoptimized benchmarks run up to 114× slower (55× more memory). Pathological cases like the Derived Value Invalidation (DVI) benchmark, which creates a deep dependency graph, are 20× slower in time but suffer **900**× higher memory use due to $O(N^2)$ GC pressure. Even a production-scale event simulator (LSim) is 7× slower than it could be.

The canonical pattern suffering from this tax is the accumulation loop, common in both `loop/recur` forms and `reduce` folds:

```
;; Loop-based accumulation
(loop [acc {}]
  ...
  (recur (assoc acc k v)))

;; Fold-based accumulation
(reduce (fn [acc x] (conj acc ...))
        [] coll)
```

In each iteration, the call to `assoc` or `conj` allocates a new persistent map or vector, immediately discarding the previous version. The key insight is that if the old `acc` is used for nothing other than producing the new `acc` (i.e., it is unique at its point of death), the update can be performed in-place.

Existing solutions are not directly transferable to a language like FOL. Perceus/FBIP [7] and its relatives [6] rely on ownership and uniqueness types, which are absent in our dynamically-typed language. Clojure [5] provides a manual, unsafe escape hatch: the programmer can explicitly opt into `transient` data structures and `assoc!`-style in-place updates, but misuse can lead to corrupted data and spooky action at a distance. Furthermore, FOL supports live redefinition of functions, meaning the semantic assumptions baked into an optimized loop might be invalidated *while that code is running*.

This paper presents a fully automatic and sound system for converting persistent accumulation loops to use in-place transients. Our contributions are:

1. **Uniqueness-at-death by usage classification.** A flow- and tail-position-sensitive classification of accumulator uses (`:recur-update`, `:exit-bare`, etc.) that establishes when a persistent accumulator may be converted to an in-place transient—without ownership or linear types. Includes chain recognition through `->` threads, if-branches, and *reduce init-threading* (a fold that linearly threads the accumulator is itself a chain link).
2. **Alignment-by-construction between analysis and transformation.** The classifier refuses to qualify anything the rewriter cannot reach (`:recur_in_complex_context`) or that would change representation mid-loop (`:recur_reset`); per-representation *op gates* ensure converted code can never hit an unsupported operation at runtime.
3. **Name-resolution-faithful summaries.** The standard library summary table matches operators exactly the way the compiler itself resolves them (by name, with per-compilation-unit exclusions)—the analysis is provably no less sound than the language's own resolution model.
4. **Soundness under live redefinition without world-age infrastructure.** Converted regions compile to guarded dual paths; validity cells register at load time indexed by assumed names; definitions notify at execution time and invalidate exactly their dependents. Guards measure as free.
5. **An honest empirical cost model.** We quantify when transient conversion is profitable. Wrapper transients (O(n) boundaries) win only at large ops-per-boundary; our edit-tagged transients (O(1)/O(32) boundaries) extend the win to small-N accumulations and read-heavy folds. We validate this with benchmarks, including a case with zero convertible sites and an end-to-end system whose hot loop is bounded by factors this technique does not address.

Our evaluation shows speedups of 4.1–7.6× on accumulation-bound microbenchmarks (§7), byte-identical results on a complex event simulator (§7), and negligible overhead when the optimization is disabled or inapplicable (§7).

2 Background and a Motivating Walk-through

FOL in one column. FOL is a Lisp-1 dialect transpiled to Common Lisp. Its core data structures are persistent. Vectors are 32-way tries with a mutable tail for efficient appends, similar to Clojure's. Dictionaries are Hash Array Mapped Tries (HAMTs) [1]. `(loop [bindings] ... (recur new-vals))` provides tail-call-optimized looping. The `->` macro threads an expression as the first argument to a series of forms. `reduce` applies a function to an accumulator and successive items from a collection. The compiler transpiles forms one by one. Calls to standard library functions are resolved by name at emit time against a known list, a fact that is load-bearing for our analysis (§3.3).

Worked Example. Consider `lh-pop-batch`, a function from the LSim event simulator. It builds up a new priority queue `acc` by repeatedly `conj`'ing items, then returns it inside a tuple literal `[acc q]`.

Original Code:

```
(defn lh-pop-batch [q n]
  (loop [acc [] i 0]
    (if (< i n)
      (let [[v new-q] (pop q)]
        (recur (conj acc v) (inc i)))
      [acc q])))  ; Exit with acc in a
          literal
```

Each call to `conj` creates a new persistent vector, copying the path from the root. Our analysis identifies `acc` as a candidate. It verifies that `acc` is freshly allocated (`[]`), never escapes the loop, and its only uses are being updated in the `recur` call (a sanctioned update) and being returned in a literal at the exit (a sanctioned exit).

Converted Code (Simplified):

```
(defn lh-pop-batch [q n]
  (if (is-valid? *lh-pop-batch-region-1*)
    ;; Fast path
    (loop [acc-t (transient []) i 0]
      (if (< i n)
        (let [[v new-q] (pop q)]
          (recur (conj! acc-t v) (inc i)))
        [(persistent! acc-t) q]))
    ;; Slow/fallback path
    (loop [acc [] i 0]
      ... ; Original code
      )))
```

The compiler emits a dual-path version. The fast path initializes a `transient` vector. The update uses the mutating `conj!` operation. At the exit, `persistent!` converts the transient back to a persistent vector in O(1) time before embedding it in the result. A guard, `is-valid?`, checked once on entry, ensures this transformation is safe. If a function like `conj` were redefined, the guard would fail, and execution would transparently use the original, safe code path.

Clojure Transient Recap. Clojure's transients provide a mutable view over a persistent collection. Updates are applied in-place to the transient version. Reads are problematic: a read from a transient may or may not see an in-place update, depending on the representation. "Wrapper" transients forbid reads entirely. "Edit-tagged" transients, which we use, support reads by navigating a mix of original and copied-on-write nodes, making them suitable for a wider range of algorithms, like read-heavy folds.

3 The Analysis

Our goal is to prove *uniqueness-at-death*: the accumulator value at each iteration is used for no other purpose than to produce the value for the next iteration, and the final value does not share its identity with any other live object. This is a stronger property than standard escape analysis, which only determines if a value *may* escape a region.

3.1 Two Properties: Escape vs. Uniqueness-at-Death

- **Escape (May-analysis):** An object *escapes* if its lifetime may exceed its static scope. Standard escape analysis tracks this to enable stack allocation.
- **Uniqueness-at-Death (Must-analysis):** An object is *unique at death* if, at the point of its last use, no other aliases to it exist. This requires:
  1. **Freshness:** The initial value is a fresh allocation not aliased elsewhere.
  2. **No-aliasing:** Inside the loop, no new aliases are created that outlive the current iteration (e.g., by storing the accumulator in a global or captured variable).
  3. **Last-use:** The `recur` or `reduce` update is the final use of the value in the current iteration.

If these hold, the object's identity is not observable, and in-place mutation is safe.

3.2 Tier-1 Summaries: The Effect Lattice

To analyze the behavior of functions called within the loop, we use a summary-based analysis. Tier-1 summaries cover the standard library. Each summary classifies a function's effect on its arguments, particularly the accumulator. We define an effect lattice:

`:none` < `:invoked` < `:shared-with-result` < `:retained`

- `:none`: The argument is not used.
- `:invoked`: The argument is called as a function but not retained.
- `:shared-with-result`: The argument may be part of the function's return value (e.g., `(identity acc)`).
- `:retained`: The argument escapes to a persistent location (e.g., `(add-watch! acc ...)`). This is a disqualifier.

We hand-verified 75 summaries for core functions. A key insight is the **root-level convention**: persistent operations like `assoc` and `conj` are summarized as returning a fresh root that may reference its (unchanged) children. Because our transient protocol is copy-on-write and never mutates elements, this summary is sound for all clients, including transient ones.

3.3 Name-Resolution Faithfulness

A summary for `assoc` is useless if the user can shadow it. In FOL, `fol.core` exports nothing. A user-package `assoc` might refer to a local variable or a different library's function. Our compiler resolves calls to standard library functions by checking if the function name exists in a special list (`+standard-fol-functions+`) and is not locally shadowed.

Our analysis mirrors this exactly. The Tier-1 summary table is consulted only for names resolved in the same way, using per-compilation-unit exclusions (*file-function-defs*) to account for local definitions. This leads to a simple but powerful guarantee:

Lemma 3.1 (Name-Resolution Faithfulness)**.** *Any program for which the analysis misjudges a function's effect (by applying a summary to the wrong function) is a program where the compiler itself would have resolved the name to the same, wrong function. The analysis is no less sound than the language's own resolution model.*

3.4 The Classifier

The classifier walks the loop body, assigning a usage kind to every appearance of the accumulator candidate. It is defined by a grammar of sanctioned uses. Key rules include:

- **Tail-position propagation:** Tail position is tracked through `if`, `do`, `let`, `case`, and `cond`. An accumulator in tail position is a candidate for an exit.
- **Literals:** An accumulator inside a literal (`[acc]`, `#{acc}`) is a sanctioned exit if the literal itself is in a tail/exit position.
- **Recur handling:** An accumulator in a `recur` argument position is classified as `:recur-update` if it's part of a sanctioned update chain.
- **Reads:** An accumulator in argument position 0 of a whitelisted read-only function (e.g., `get`, `count`) is classified as `:read-ok`. Using an accumulator as a *key* in a map lookup, for example, would be a disqualifier, as it relies on value semantics that mutation would violate.
- **Disqualifiers:** Any non-sanctioned use (e.g., passing to an unknown function, being stored in a closure) disqualifies the candidate. Two special disqualifiers ensure alignment with the rewriter:
  - `:recur_reset`: The accumulator is reset to a new value not derived from the old one (e.g., `(recur {} ...)`). This would require changing representation mid-loop.
  - `:recur_in_complex_context`: The update expression is too complex for the rewriter to transform (e.g., `(recur (if (foo) (assoc acc k v) acc))`).

3.5 Chains

Update chains are sequences of operations that thread the accumulator linearly. The classifier recognizes:

- **Direct calls:** `(assoc acc k v)`
- **`->` threads:** `(-> acc (assoc k v) (dissoc k2))`

- **If-branched chains:** `(if test (assoc acc k v) (dissoc acc k2))` where both branches are valid chains.
- **Reduce init-threading:** A `reduce` form is itself a valid chain link if its accumulator is threaded linearly. For `(reduce f init-val coll)`, if `init-val` is a chain on the outer accumulator, and the reducing function `f` qualifies under special fold rules (its own accumulator is used linearly and it doesn't capture the outer one), the entire `reduce` is treated as a single `:update` link. This allows optimizing loops that contain inner reduction loops, a common pattern.

3.6 Formal Core

We formalize the analysis with a judgment $\Gamma \vdash e : acc \Rightarrow U$, which states that in context Γ, expression e results in a set of uses U for accumulator *acc*.

Accumulator Use Judgment: $U ::= $ `:recur-update`(e') | `:exit-bare` | `:exit-in-literal` | `:read-ok` | ...

Qualification Rule (Simplified):

$$\frac{\Gamma \vdash \text{init fresh} \quad \forall u \in \text{uses}(\text{body}, acc), u \in \text{SanctionedUses}}{\text{Qualifies}(\texttt{loop [acc init] body})}$$

This leads to our main theorem, stating that the transformation preserves observational equivalence.

Theorem 3.2 (Soundness of Conversion)**.** *If every use of an accumulator* `acc` *is sanctioned by the classifier, then replacing the persistent operations on* `acc` *with their transient counterparts and inserting* `persistent!` *at all exits yields a program that is observationally equivalent to the original.*

The proof proceeds by bisimulation against a store-based semantics. The ownership tokens used in our transient implementation (§4.3) discharge the obligations regarding interior node sharing. A detailed proof sketch is available in Appendix A.

4 The Transformation and the Transient Representation

4.1 The Rewriter

Once the classifier qualifies a loop, the rewriter transforms the code.

1. **Initialization:** The initial binding is wrapped in a `(transient ...)` call.
2. **Update Chains:** At each `recur` site, the update expression is transformed into its "bang" equivalent (e.g., `assoc` → `assoc!`).
3. **Exits:** Every exit point (bare, in a chain, or in a literal) is wrapped in `(persistent! ...)`.
4. **Modes:** The rewriter operates in `:loop` or `:reduce` mode. In `:reduce` mode, the transient is threaded through the reduction without intermediate `persistent!` boundaries, enabling full optimization of folds.

The transformation preserves structure sharing for any subtrees that are not modified.

4.2 Per-representation Op Gates

Not all persistent collections have a full-featured transient equivalent. Our rewriter enforces this with *op gates*. Before converting a loop, it collects the set of all update and read operations used on the accumulator. It then checks this set against the allowed operations for the accumulator's data type:

- **Dicts:** `{assoc!, dissoc!}` + reads (`get`, `contains?`, etc.)
- **Vectors:** `{conj!}` + reads
- **Sets:** `{conj!, disj!}` (no reads supported)

If a loop uses an unsupported operation (e.g., trying to read from a transient set), the chain is considered incompatible, and the loop is left unconverted. This ensures at compile time that converted code can never hit an unsupported operation.

4.3 Edit-tagged Transients

Our implementation is based on edit-tagged transients, inspired by Clojure.

- **HAMT (Dicts/Sets):** Nodes in the trie are tagged with an "edit" token. When a transient is created, a new token is minted. On a write, `ensure-editable` checks the node's tag. If it matches the transient's token, the update is in-place. If not (the node is from a prior persistent structure), a copy is made, tagged, and then mutated. `persistent!` freezes the transient by clearing the token, making all its nodes read-only.
- **Vectors:** A transient vector has an owned, mutable tail array. `conj!` appends to this tail. If the tail overflows, a new one is allocated, and the path is updated via the same copy-on-write mechanism as HAMTs. This makes appends O(1) on average, with an O(32) cost at boundary crossings.
- **Transient-aware Reads:** A critical detail is that a standard persistent lookup function will **silently misread** an edited transient, as it doesn't know to check for copied-on-write nodes. Our read operations (`get`, `count`, etc.) are transient-aware: they accept both persistent and transient collections and use the correct internal lookup function (`hamt-get-transient`, `%transient_vec_t_ref`) when a transient is detected.

4.4 Dynamic-extent Closures

As a related optimization, the analysis infrastructure is also used to identify closures with dynamic extent. A literal `fn` passed to a verified non-retaining, eager higher-order function (like `map` or `reduce`) can be stack-allocated. This optimization is also world-guarded, as a redefinition of `map` to be retaining would otherwise cause a dangling pointer to a stack object.

5 Soundness under Live Redefinition

Threat Model: A converted loop bakes in the semantic assumptions from our Tier-1 summaries (e.g., that `assoc` is a pure function). A subsequent runtime definition, like `(defn assoc [c k v] (log! c) ...)` or `(defmethod assoc :around ...)` invalidates this assumption.

Mechanism:

1. **Dual-path Emission:** The compiler emits both the optimized (fast) and original (slow) code paths for each converted region.
2. **Region Registration:** At load time, each converted region calls `(register-region)` which returns a *validity cell*. The region is registered as a dependent on every standard library function name it assumes (e.g., `assoc`, `conj`).
3. **Invalidation:** In optimizer mode, definitions for functions like `assoc` are wrapped with a call to (note-redefinition 'assoc). This function looks up all dependent validity cells for `assoc` and flips them to `invalid`.
4. **Guard Check:** The fast path is guarded by a check on the validity cell. If invalid, execution falls back to the slow path. In-flight iterations are unaffected and complete on the path they entered on, which is sound because the accumulator is only touched through compiler-owned operations within the region.

This mechanism provides precise, per-name dependency tracking. It differs from Julia's world age counter [2] in that we protect escape/uniqueness facts rather than method dispatch tables, operate at region rather than method granularity, and require no runtime recompilation. A panic switch to disable all optimizations serves as a first line of defense.

In a batch compilation mode (*sealed_world*), we can assume the world is closed, and no guards are emitted.

6 Implementation

The system is implemented in ~2,430 lines of Common Lisp, spread across `summaries.lisp`, `escape-analysis.lisp`, `world.lisp` and hooks in the compiler. It is covered by 217 new tests, bringing the project total to 3,273 (100% coverage).

A key methodological tool was the **audit mode**. This mode enables the analysis to run against any codebase, collecting counts of convertible sites, disqualification reasons, and op-gate statistics, but without emitting any transformed code. We used this to size the opportunity and guide precision improvements before writing a single line of rewriter code. For example, the audit on LSim showed 17 loops with 9 accumulator candidates, all of which qualified after we implemented `reduce` init-threading and literal-exit support. The audit also showed that Tier-1 summaries covered 54.8% of 542 call sites in LSim's hot loop, with the remainder being genuine user code.

7 Evaluation

We evaluate our work by answering six research questions. All microbenchmarks were run on a Ryzen 9 5900X with SBCL 2.6.0, reporting the best of 3 runs. The baseline is the `v0.1.1` tag of the FOL compiler, before this work.

RQ1: Correctness. Our system passes 3,248 tests, including specific tests for ownership semantics (e.g., ensuring `persistent!` freezes a transient). The LSim event simulator produces byte-identical output (over full event histories of 876 and 281,248 gate evaluations) when run with the baseline and the optimized compiler.

RQ2: Speedup on accumulation-bound code. We measured significant speedups and memory reduction on synthetic benchmarks designed to isolate the accumulation pattern.

Table 1. Performance on accumulation-bound microbenchmarks.

Workload	v0.1.1	Optimized	Time	Memory
dict loop, 200k assocs	224.6ms 223MB	55.2ms 26MB	**4.07**×	**8.7**×
vector loop, 1M conjs	309.9ms 352MB	41.0ms 32MB	**7.56**×	**11.0**×
reduce, 500k conjs	324.1ms 359MB	48.8ms 40MB	**6.64**×	**9.1**×
small-N (3 assocs/boundary)	130.9ms 127MB	116.9ms 99MB	**1.12**×	1.3×
small-N with reads	174.8ms 151MB	135.3ms 99MB	**1.29**×	1.5×

The results show dramatic improvements for large loops. Even for small-N loops with only 3 operations per boundary, our edit-tagged transients provide a modest win, where simpler wrapper transients would have lost performance.

RQ3: Ablation: wrapper vs edit-tagged transients. To validate our cost model, we compared our edit-tagged implementation to a hypothetical wrapper-based one (simulated by disabling reads). Edit-tagged transients improve speedups from 2.30× to 4.14× for dictionaries and 5.60× to 7.52× for vectors on large loops. For small-N loops, they turn a ~1.0× result into a 1.16× win and enable conversion of read-heavy loops that were previously impossible (unconvertible → 1.38×). This confirms that lowering the boundary cost from O(n) to O(1)/O(32) is critical.

RQ4: End-to-end (LSim event simulator). On two LSim workloads, we measured wall-clock speedups of 1.01× and 1.04× (mean of 5 runs), with a 1.10× reduction in allocation (−180MB/run) on the larger workload. The modest speedup is explained by our cost model: the converted loops in LSim

are small-N (2–3 operations per accumulation), and the simulator's hot loop remains bounded by other factors, namely leftist-heap node churn and generic function dispatch, which our technique does not address. The analysis automatically converted 2 loops and 3 folds, correctly identifying a nested `reduce` via init-threading.

RQ5: Cost when off / inapplicable. When the optimization flag is off, performance is identical to the v0.1.1 baseline within measurement noise (e.g., 225.2ms vs 224.6ms), and emitted code is byte-stable by test. On LSim, enabling world guards had no measurable impact on performance (1.02× with vs. without, within noise).

RQ6: Live invalidation. We tested the invalidation mechanism by running a converted loop, redefining `assoc` to include side effects, and re-running the loop. The same function object correctly fell back to the slow path, producing identical results to the original, with world-stats counters confirming that only the `assoc`-dependent regions were invalidated.

Threats to Validity. Our implementation is for a single language on a single machine. The microbenchmarks are synthetic. LSim represents one real-world application.

8 Discussion and Limitations

DVI now converts. With the addition of Tier-2 interprocedural summaries, the DVI benchmark is now successfully converted. The analysis correctly infers that the 'add-item' helper function is pure and uses its accumulator linearly, qualifying the main loop for transient conversion. This yields a **7.27× speedup** and a **67.5× reduction in memory usage**, eliminating the pathological GC behavior that motivated this work.

Profitability. Our small-N wins are modest, bounded by the constant-factor costs of the transient machinery. A profitability heuristic that only enables the optimization when an accumulator's size exceeds a certain threshold at runtime is possible but currently unimplemented.

Completeness. Our transient sets are "wrapper" style and do not support reads. Transient vectors only support `conj!`. The op-gate mechanism makes these completeness limitations, not correctness issues; the system safely refuses to convert loops that use unsupported operations.

This work represents the first three rungs of an "invalidation ladder," including inferred user-function summaries. Future work could add monotonic redefinitions (e.g., adding a method to a generic function that doesn't affect the existing dispatch) and tripwire-based recompilation.

9 Related Work

Perceus/FBIP. Reinking et al. [7] and Lorenzen & Leijen [6] introduced compile-time lifetime analysis and ownership types to enable functional-but-in-place updates. Our work achieves a similar goal but in a dynamically typed language without requiring any type system annotations from the programmer, and we additionally handle the complexities of a live, redefinable image.

Clojure. Our transient protocol builds on the edit-token design pioneered by Hickey in Clojure [5]. The key difference is that Clojure's transients are a manual, opt-in, and potentially unsafe mechanism, whereas our system is fully automatic and sound. Our work draws on the foundational literature of persistent data structures by Bagwell [1] and Steindorfer & Vinju [9].

World Age. Julia's world age mechanism [2] invalidates code based on changes to method tables. As noted in §5, our work differs in what it protects (uniqueness facts vs. dispatch), its granularity (region vs. method), and its invalidation mechanism (precise dependency tracking vs. a monotonic counter), avoiding the need for runtime recompilation.

Escape Analysis. Our work is in the lineage of escape analysis [3, 4, 8], but our client is different. Instead of stack allocation, we target uniqueness-at-death to enable mutation of persistent data structures. Unlike classic interprocedural analyses, our boundary policy relies on trusted, name-resolution-faithful summaries of a standard library, which is a more practical approach in a dynamic language where whole-program analysis is often infeasible.

Linear and Uniqueness Types. The concept of unique, unaliased values dates back to linear and uniqueness types [11]. Systems like Clean and Rust use type systems to enforce these properties. Our work provides the benefits of uniqueness for a specific, common pattern without imposing any type system obligations on the programmer.

Deforestation. The "reduce init-threading" concept (§3.5) can be seen as a form of deforestation or fusion [10], where an intermediate data structure (the result of the inner reduce) is eliminated by threading the accumulator through it.

10 Conclusion

We have presented a system for automatically and safely converting allocation-heavy accumulation loops into in-place updates in a dynamic, persistent-first language. By combining a novel uniqueness-at-death analysis, a rewriter aligned by construction, and a lightweight guarding mechanism for soundness under live redefinition, we achieve significant performance gains on targeted workloads. Our honest cost model and negative results clearly delineate the boundaries of the technique's effectiveness, showing that edit-tagged transients are crucial for profitability and paving the way for future work in interprocedural summary analysis.

References

[1] Phil Bagwell. 2001. *Ideal hash trees.* Technical Report INFO-TR-01-004. EPFL.

[2] Julia Belyakova, Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B Shah. 2020. A JIT compiler for Julia. In *Proceedings of the ACM on Programming Languages*, Vol. 4. ACM New York, NY, USA, 1–29.

[3] Bruno Blanchet. 2003. Escape analysis for object-oriented languages. Application to Java. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 25, 6 (2003), 713–775.
[4] Jong-Deok Choi, Manish Gupta, Mauricio Serrano, VC Sreedhar, and Sam Midkiff. 1999. Escape analysis for Java. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 1–19.
[5] Rich Hickey. 2009. The Clojure Programming Language. https://clojure.org/.
[6] Nikolaj Sidorenco Lorenzen and Daan Leijen. 2023. Functional but in-place: effect-based and uniqueness-based borrowing. In *Proceedings of the 8th ACM SIGPLAN International Symposium on Principles and Practice of Declarative Programming*. 1–14.
[7] Alex Reinking, Kostis Sagonas, and Daan Leijen. 2021. Perceus: Garbage free reference counting with reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 764–779.
[8] Lukas Stadler, Christian Wimmer, and Hanspeter Mössenböck. 2014. Partial escape analysis and scalar replacement for Java. In *Proceedings of the 12th annual IEEE/ACM international symposium on code generation and optimization*. 1–11.
[9] Michael J Steindorfer and Jurgen J Vinju. 2015. Optimizing hash-array mapped tries for fast and lean immutable collections. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*. 783–800.
[10] Philip Wadler. 1990. Deforestation: transforming programs to eliminate trees. *Theoretical computer science* 73, 2 (1990), 231–248.
[11] Philip Wadler. 1990. Linear types can change the world!. In *IFIP TC 2 Working Conference on Programming Concepts and Methods*.

A Proof Sketch for Theorem 1 (Soundness of Conversion)

The proof of Theorem 1 proceeds by bisimulation. We define an operational semantics for both the original (Source) and transformed (Target) programs and show that they are observationally equivalent.

A.1 Semantics

We model program execution with a state $\langle e, \rho, \sigma \rangle$, where e is the expression to evaluate, ρ is an environment mapping variable names to locations, and σ is a store mapping locations to values.

- **Values:** $v ::= l | n | \ldots$ where l is a location in the store.
- **Store:** $\sigma : l \mapsto v$. Collections are represented as heap-allocated objects. A persistent map is Map(root). A transient map is TMap(root, tok), where tok is a unique ownership token.
- **Source Semantics (Persistent):** An update `(assoc m k v)` evaluates to a *new* location l' pointing to a new map. The store σ is extended to $\sigma' = \sigma \cup \{l' \mapsto \text{Map(new\_root)}\}$. The original map at $\rho(m)$ is unchanged.
- **Target Semantics (Transient):** An update `(assoc! m k v)` *mutates* the map at $\rho(m)$. If the node to be changed has the correct ownership token, it is modified in-place. Otherwise, a copy is made, tagged with the token, and then mutated. The store σ is updated to σ' where the object at $\rho(m)$ is modified.
  `(persistent! m)` "freezes" the transient by creating a new persistent map and clearing the token.

A.2 Bisimulation Relation

We define a bisimulation relation $\approx$ between a Source state $S = \langle e_S, \rho_S, \sigma_S \rangle$ and a Target state $T = \langle e_T, \rho_T, \sigma_T \rangle$. We write $S \approx T$ if:

1. e_S is syntactically identical to e_T outside the converted loop. Inside the loop, e_S and e_T correspond via the transformation rules (e.g., `assoc` vs. `assoc!`).
2. The environments ρ_S and ρ_T map the same variables to corresponding locations, except for the accumulator *acc*.
3. The stores σ_S and σ_T are equivalent for all locations reachable from ρ_S and ρ_T, according to a value relation $\approx_v$.

The value relation $v_S \approx_v v_T$ holds if:

- v_S and v_T are identical scalars.
- v_S is a persistent collection $\text{Map}(r_S)$ and v_T is a persistent collection $\text{Map}(r_T)$, and they are structurally identical.
- v_S is the persistent accumulator $\text{Map}(r_S)$ at location l_{acc_S}, and v_T is the corresponding transient accumulator $\text{TMap}(r_T, \text{tok})$ at l_{acc_T}, and they are structurally identical (ignoring the token).

A key part of the invariant is that for the accumulator *acc*, $\rho_S(acc)$ must be the *only* reference to the map at that location in the source store. This is guaranteed by the uniqueness-at-death analysis.

A.3 Proof Sketch by Induction on Evaluation Steps

We show that if $S \approx T$ and the Source program takes a step $S \to S'$, then the Target program can take a step $T \to T'$ such that $S' \approx T'$.

Base Case: Loop Initialization.

- Source: `(loop [acc (hash-map)] ...)` evaluates to a state where $\rho_S(acc)$ points to a fresh, unaliased map m_S.
- Target: `(loop [acc (transient (hash-map))] ...)` evaluates to a state where $\rho_T(acc)$ points to a fresh transient map m_T with a new token.
- The states are related: $m_S \approx_v m_T$, and the uniqueness property for *acc* holds in the Source.

Inductive Step: Inside the Loop. We consider the evaluation of the loop body.

- **Case 1: Read operation `(get acc k)`.**
  - Source: Reads from the persistent map at $\rho_S(acc)$.
  - Target: Reads from the transient map at $\rho_T(acc)$. Our transient-aware read operations are designed to correctly interpret the mix of original and copied-on-write nodes.

  - Since the structures are equivalent, they return equivalent values. The resulting states S' and T' are still in relation.
- **Case 2: Update operation (`recur (assoc acc k v)`).**
  - Source: `(assoc acc k v)` evaluates to a *new* location l'_S with a new map m'_S. The old map at $\rho_S(acc)$ is now garbage (by the last-use property from our analysis). The `recur` rebinds acc to l'_S. The uniqueness property is maintained for the new accumulator value.
  - Target: `(assoc! acc k v)` *mutates* the transient map at $\rho_T(acc)$ in-place. The `recur` rebinds acc to the same location $\rho_T(acc)$.
  - The new map m'_S and the mutated map at $\rho_T(acc)$ are structurally equivalent. The resulting states S' and T' are in relation, and the uniqueness invariant is maintained.
- **Case 3: Disqualifying use (`some-func acc`).**
  - This case is ruled out by the classifier. If `some-func` could retain an alias to acc, the uniqueness property would be violated in the Source program, and the in-place update in the Target program would become observable, breaking the equivalence. The analysis prevents this.

Inductive Step: Loop Exit.

- Source: The loop exits, returning the final persistent accumulator m_S.
- Target: The loop exits. The expression is wrapped with `(persistent! acc)`, which evaluates the transient m_T to a new persistent map m'_T.
- Since $m_S \approx_v m_T$ before the exit, the final results m_S and m'_T are structurally identical and thus observationally equivalent to the rest of the program.

Conclusion. By induction, for any number of evaluation steps, the Source and Target programs remain in the bisimulation relation. Since the initial states are related and the relation preserves program termination and final values, the transformed program is observationally equivalent to the original. The soundness of the transformation thus rests on the guarantees provided by the uniqueness-at-death classifier.